

Workflow Optimization

Proving the Ignition redesign makes real daily work measurably faster. Task inventory, click-path analysis before vs after, the Action Queue as a workflow engine, Client Workspace operational flows, speed mechanics, and the telemetry plan that verifies every projection.

- ACTION QUEUE
- CLIENT WORKSPACE
- CTRL+K PALETTE
- 55% STEPS / DAY
- ~98 MIN SAVED / SHOP / DAY

AUTHOR	workflow-optimization — Workflow Agent
DATE	2 July 2026
SCOPE	Dashboard redesign + unified Client Workspace — React web app
BASIS	Code audit of src/SpareParts.Web.React/wwwroot/js/views (74 registered screens)

Contents

Every claim in this report is grounded in the current code of the React web app. Where a number is a projection, the assumption is stated and Section 8 defines the telemetry event that will confirm or refute it.

01	Executive Summary	Where daily time is lost today, and the projected gains	p. 3
02	Task Inventory	The 10 highest-frequency daily tasks — who, how often	p. 4
03	Click-Path Analysis — Before vs After	Step counts derived from the real navigation structure	p. 5–6
F1–F3	Flow Figures	Swimlane diagrams: counter sale · receive payment · chase overdue	p. 7–8
04	The Action Queue as a Workflow Engine	Prioritization, lifecycle, deep-link contract	p. 9–10
05	Client Workspace Operational Flows	Payment, invoice-from-context, collection worklist, follow-ups	p. 11
06	Speed Mechanics	Ctrl+K grammar, keyboard-first counter sale, barcode, recents	p. 12
07	Cross-Device Workflow	Phone vs desktop split; offline notes for the counter	p. 13
08	Adoption & Rollout	Side-by-side period, telemetry, success metrics with targets	p. 14
F5	A Day in the Life — Before vs After	Timeline of administrative overhead across one shop day	p. 15

Figures

FIG.	TITLE	PAGE
F1	Counter sale to a known client — swimlane, today vs Ignition	7
F2	Minutes saved per task per day (projected)	7
F3	Receive a payment against balance — swimlane	8
F4	Chase overdue receivables — swimlane	8
F5	Action Queue item lifecycle — state diagram	10
F6	A day in the life — administrative overhead timeline	15

01 Executive Summary

SpareParts today is **feature-rich but navigation-heavy**. The web app registers **74 screens** in `screen-registry.js`, and the work of a single business moment — one client, one sale, one payment — is scattered across many of them. A counter sale lives in `invoices-view.js`, the client's balance in `contacts-view.js`, their overdue buckets in `customer-aging-view.js`, their quotes in `quotes-view.js`, their reminders in two different screens, and payment receipt has **no direct flow at all** — the invoice list offers only *View Profit / Send Invoice / Reminder* actions, so a payment received after invoicing means a manual journal entry in Accounting.

Where time is lost today (from the code, not from opinion)

- **Fragmented client context.** Contacts is a read-only two-column list capped at 100 rows with no search, no row click-through, and no create form. Answering “what does Elie owe and why?” takes three screens.
- **Dropdown-driven data entry.** The POS customer picker is a plain `<select>` fed by `/api/customers?page=1&pageSize=100`; the part picker is a `<select>` over the *entire catalog* (`api.list("/api/parts")`). Every invoice line is a scroll-and-hunt exercise.
- **Dead-end search.** The dashboard “Smart Search” input discards what you type — its Enter handler navigates to Inventory without passing the query — so every lookup is typed twice.
- **An action queue that can't act.** `buildActionQueue()` already computes 6 real business signals (negative P&L, unpaid receivables, red stock segments, currency-squeezed margins, failed messages, no-sales-yet), but caps at 4 items, links only to whole screens with no entity context, and forgets everything on refresh.
- **Blind messaging.** Payment reminders fire with no preview; delivery failures surface only as a filtered list on the dashboard, disconnected from the client they belong to.

799 → 358

UI STEPS PER SHOP-DAY

-55%

STEP REDUCTION

~98 min

SAVED PER SHOP PER DAY

95s → 30s

COUNTER SALE, 2 LINES

What the redesign changes

The Dashboard becomes a work tool. The Action Queue is promoted from a 4-item hint list to the organizing principle of the screen: every item is scored by *severity × money at stake*, has a lifecycle (appears → acted → resolved/dismissed), and deep-links into the exact action — not just a screen.

One screen per client. The Client Workspace merges Contacts + Customer Aging + per-screen invoice/quote/request lookups into a single customer-360 view where a payment can be received, an invoice raised, a quote drafted, and a WhatsApp reminder previewed and sent — without navigating away.

Keyboard-first speed. A global `Ctrl K` palette with a terse grammar (`inv eli` → new invoice for Elie) plus barcode-wedge entry collapses the two highest-frequency tasks — counter sales and stock checks — to seconds.

Bottom line: across the 10 highest-frequency tasks, projected daily savings are ~47 min for counter staff, ~43 min for the owner, and ~8 min on quoting — roughly **1.6 hours per shop per day**, or one full working month per year. Section 8 defines the time-on-task telemetry that will verify these numbers during the side-by-side rollout; no projection ships unmeasured.

02 Task Inventory

The ten tasks below account for the overwhelming majority of daily interactions with the app. Frequencies are planning assumptions for a typical single-location spare-parts shop (owner + 1–2 counter staff); the telemetry plan in Section 8 replaces these assumptions with measured counts during rollout.

#	TASK	PRIMARY ROLE	FREQ / DAY	WHERE IT LIVES IN TODAY'S UI
T1	Morning review — “what needs my attention today?”	Owner	1	Dashboard + Accounting + Customer Aging + Stock + WhatsApp (KPI cards link to whole screens)
T2	Counter sale to a known client (typical 2 lines)	Counter staff	25	POS / Sales — create panel with customer <code><select></code> and full-catalog part <code><select></code>
T3	Counter sale to a new client (create client first)	Counter staff	3	No create form in Contacts; detour via Management, then back to POS (lookup list loads once on mount)
T4	Receive a payment against balance (after invoicing)	Owner / cashier	6	No direct flow — invoice rows have no “receive payment” action; manual journal in Accounting
T5	Chase overdue receivables (batch of ~5 clients)	Owner	1	Customer Aging (buckets only) cross-referenced against POS/Sales invoice search, per client
T6	Check if a part is in stock + price it (phone/walk-in ask)	Counter staff	30	Inventory screen; dashboard search box discards the typed query on Enter
T7	Create quote → convert to invoice → send	Owner / sales	4	Quotes screen (convert needs re-click confirm), then POS/Sales screen to find and send the new invoice
T8	End-of-day cash check	Owner	1	Dashboard cash KPI + Accounting tabs; no register/day-close concept — manual arithmetic
T9	Follow up a part request (quote-ready or source gap)	Counter staff	5	Part Requests screen → Inventory to verify stock → WhatsApp to message the customer
T10	Send a WhatsApp payment reminder (ad hoc)	Owner	4	Customer Aging “Send Reminder” (no preview) or per-invoice “Reminder” in POS/Sales; delivery status only on dashboard

Why these ten

They map one-to-one onto the signals the existing dashboard already computes in `buildActionQueue()` — receivables (T4, T5, T10), sales flow (T2, T3, T7), stock health (T6, T9), daily P&L and cash (T1, T8), and message failures (T10). The redesign does not invent new work; it shortens the paths to work the system already knows about.

Role load at a glance (executions/day × current path length)

Counter staff carry the volume: T2 + T6 alone are ~55 executions/day of scroll-heavy dropdown UI. **The owner** carries the fragmentation: T1, T4, T5, T8, T10 each span 2–4 screens with manual cross-referencing. The optimization targets are therefore different — *keystrokes* for the counter, *context assembly* for the owner — and Sections 5–6 address them separately.

03 Click-Path Analysis — Before vs After

Method

A **step** is one user interaction: a navigation (sidebar click + screen load), a click/tap, a scroll-hunt inside a dropdown, or a typed field committed with Enter. Timing uses keystroke-level constants: navigation + load ≈ 6 s, dropdown scroll-find ≈ 8 s, click ≈ 2 s, typed field ≈ 4 s, palette command ≈ 2 s, plus reading time where stated. “Before” paths are traced from the actual view code; “after” paths from the Ignition interaction specs (Reports 01–03).

#	BEFORE — TODAY’S PATH (TRACED FROM CODE)	AFTER — IGNITION PATH	STEPS	TIME
T1	Dashboard renders 8 KPI cards + 8 bottom stats + 9 snapshot cards; interpreting them requires visiting Accounting, Customer Aging, Reorder, WhatsApp. Queue caps at 4 items, links to whole screens. 6 screens • 22 steps	Single dashboard: KPI band + ranked Action Queue (severity × money). Read queue top-to-bottom; open 1–2 deep links that land on the exact action. 1 screen • 5 steps	22 → 5	12 → 4 min
T2	Navigate POS/Sales → open customer <code><select></code> , scroll ≤100 names → pick → per line: open full-catalog part <code><select></code> , hunt, pick, qty, price, “Add Line” (×2) → paid amount → Create Invoice. 15 steps	Ctrl+K → <code>inv eli</code> ↵ (draft opens, client set, cursor in scan field) → scan/type code ↵ → qty ↵ → scan line 2 ↵ → tender ↵ (defaults to total). 7 steps	15 → 7	95 → 30 s
T3	Contacts has no create form (read-only lists). Detour: Management workspace → create client → back to POS → customer list was loaded once on mount, so reload the screen → then run T2. ~18 steps	In the same palette flow: unknown name → “Create ‘Nabil Fakhry’ as new client” inline → phone ↵ → sale continues with the new client attached. 9 steps	18 → 9	4 min → 75 s
T4	Invoice rows expose only <i>View Profit / Send Invoice / Reminder</i> ; <code>paidAmount</code> is set at creation only. Later payment ⇒ Accounting → find party ledger → manual journal (debit cash, credit A/R) → post → verify. 12 steps	Client Workspace → Receive Payment (amount pre-filled with balance, allocation oldest-first) → confirm → receipt prints, aging pill re-colors in place. 4 steps	12 → 4	3 min → 30 s
T5	Customer Aging shows buckets but not the invoices behind them. Per client (×5): search POS/Sales, cross-reference amounts, back, “Send Reminder” blind (no preview), verify delivery on dashboard. ~21 steps	Queue item “3 invoices overdue — Garage Elie Motors” → Client Workspace payments tab shows the exact invoices; reminder pre-drafted with them listed → preview → send → “next client”. 8 steps	21 → 8	15 → 5 min

Grounded detail worth repeating: the current dashboard search box (`ck-cmd-input`) navigates to Inventory on Enter *without passing the query* — the user retypes every search. In Ignition, **Ctrl+K** answers stock/price questions inline without leaving the current screen at all (T6, next page).

Click-Path Analysis — Tasks 6–10 & Summary

#	BEFORE — TODAY'S PATH	AFTER — IGNITION PATH	STEPS	TIME
T6	Navigate to Inventory (dashboard search discards the query) → type search → Enter → scan result row for price + stock. 4-5 steps	Ctrl+K → <code>part n54 coil</code> → result card inline: price, stock by warehouse, margin flag. No navigation. 2 steps	5 → 2	20 → 8 s
T7	Quotes screen → search → open detail → "Convert to Invoice" → re-click "Confirm Convert" (warning path for non-catalog lines) → navigate POS/Sales → search the new invoice → "Send Invoice". 13 steps	Client Workspace quotes tab → open quote → Convert & Send (one action: converts, previews the WhatsApp invoice message, sends) → confirm. 6 steps	13 → 6	3 min → 60 s
T8	Dashboard cash KPI → Accounting → locate cash account across tabs → pull today's invoice paid totals from POS/Sales → manual arithmetic on paper/calculator. 10 steps	Day-Close card on dashboard: opening cash + receipts – payouts = expected drawer; variance highlighted; one click to drill into any line. 3 steps	10 → 3	8 → 2 min
T9	Part Requests screen → open row → Inventory to verify stock/price → back → WhatsApp screen to message the requester. 9 steps	Queue item "Quote-ready request" (signal already computed as <code>quoteReadyRequests</code> today) deep-links to the request with live stock and a drafted quote message. 4 steps	9 → 4	2 min → 45 s
T10	Customer Aging → search client → "Send Reminder" fires <code>accountBalanceReminder</code> with no preview → dashboard recent-messages list to check for "fail" status. 7 steps	Client Workspace messages tab → template with live balance + invoice list → edit if needed → send; delivery status lands back on the same thread. 3 steps	7 → 3	90 → 30 s

Daily summary — steps and minutes, weighted by frequency

#	TASK	ROLE	FREQ	STEPS BEFORE	STEPS AFTER	Δ STEPS/DAY	SAVED/DAY
T1	Morning review	Owner	1	22	5	-17	8 min
T2	Counter sale — known client	Counter	25	375	175	-200	27 min
T3	Counter sale — new client	Counter	3	54	27	-27	8 min
T4	Receive payment vs balance	Owner/cashier	6	72	24	-48	15 min
T5	Chase overdue (batch of 5)	Owner	1	21	8	-13	10 min
T6	Stock check + price	Counter	30	120	60	-60	6 min
T7	Quote → invoice → send	Sales	4	52	24	-28	8 min
T8	End-of-day cash check	Owner	1	10	3	-7	6 min
T9	Part-request follow-up	Counter	5	45	20	-25	6 min
T10	WhatsApp payment reminder	Owner	4	28	12	-16	4 min
Total per shop-day			80	799	358	-441 (-55%)	~98 min

By role: counter staff ≈ 47 min/day · owner ≈ 43 min/day · sales/quoting ≈ 8 min/day. Over a 6-day week that is ~9.8 hours per shop; over a year, roughly one working month recovered.

Figure 1 — Counter Sale to a Known Client

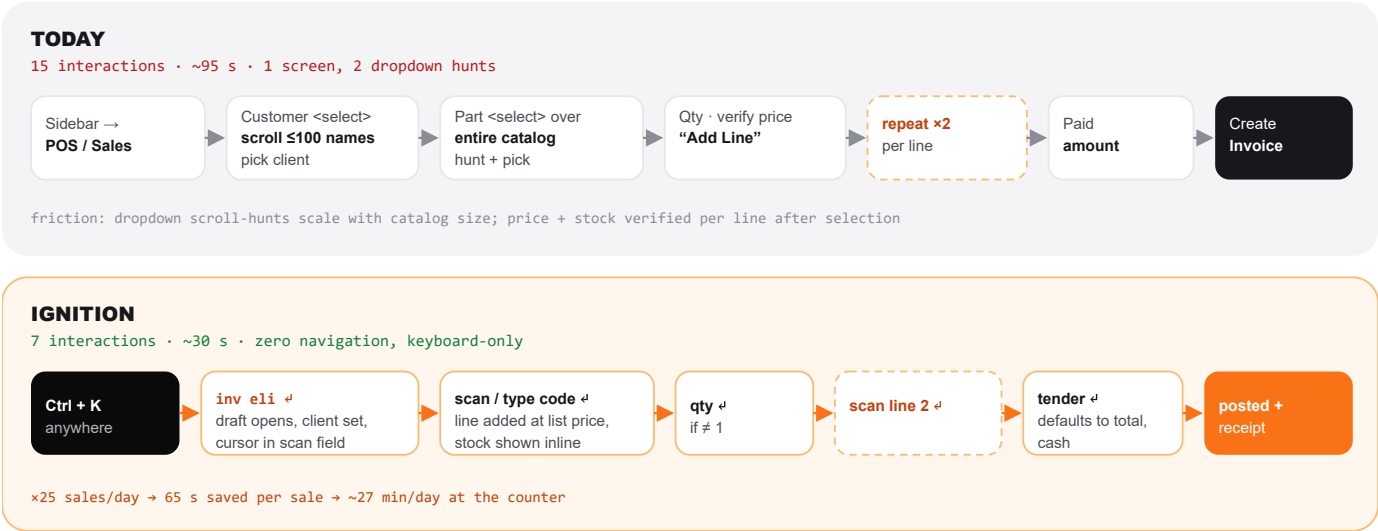


FIG 1 · Swimlane – counter sale to a known client (2 lines), traced from invoices-view.js vs Ignition keyboard-first flow.

Figure 2 — Minutes Saved per Task per Day (projected)

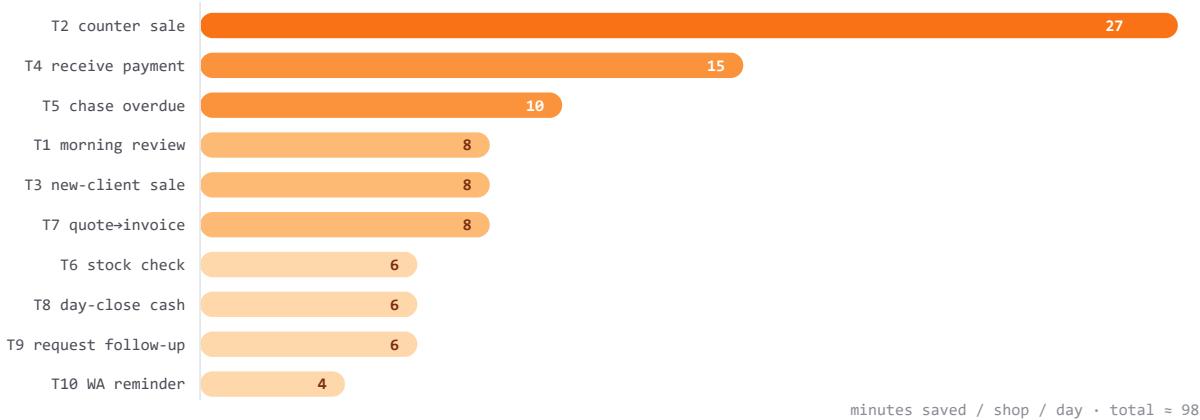


FIG 2 · Projected minutes saved per day, per task – weighted by execution frequency (Section 3 summary table).

Figure 3 — Receive a Payment Against Balance

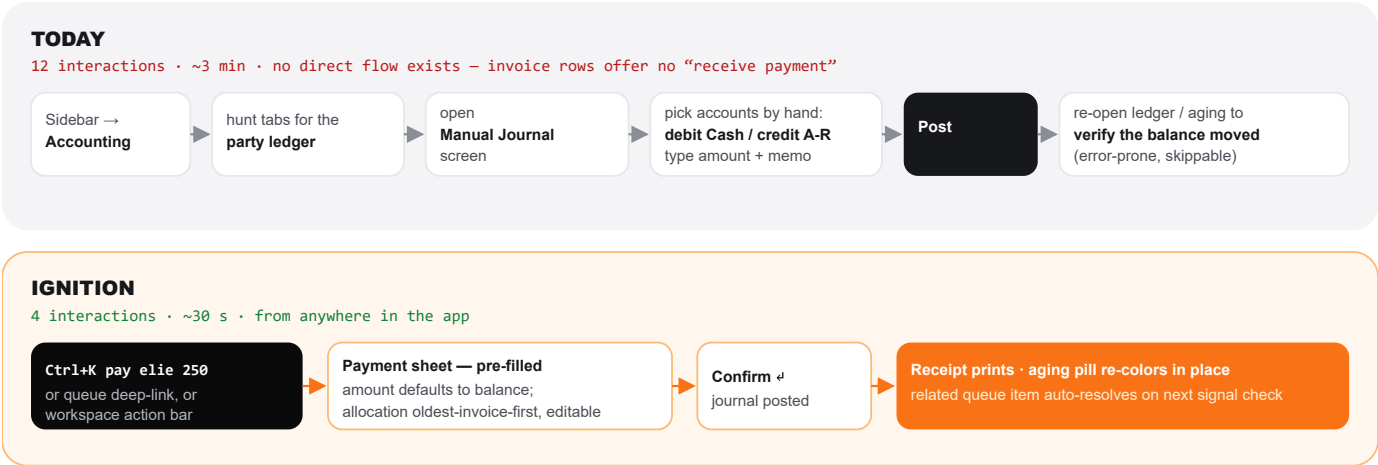


FIG 3 · Swimlane — receiving a payment after invoicing. Today’s path is a manual journal; Ignition makes it a first-class one-sheet action.

Figure 4 — Chase Overdue Receivables (5 clients)

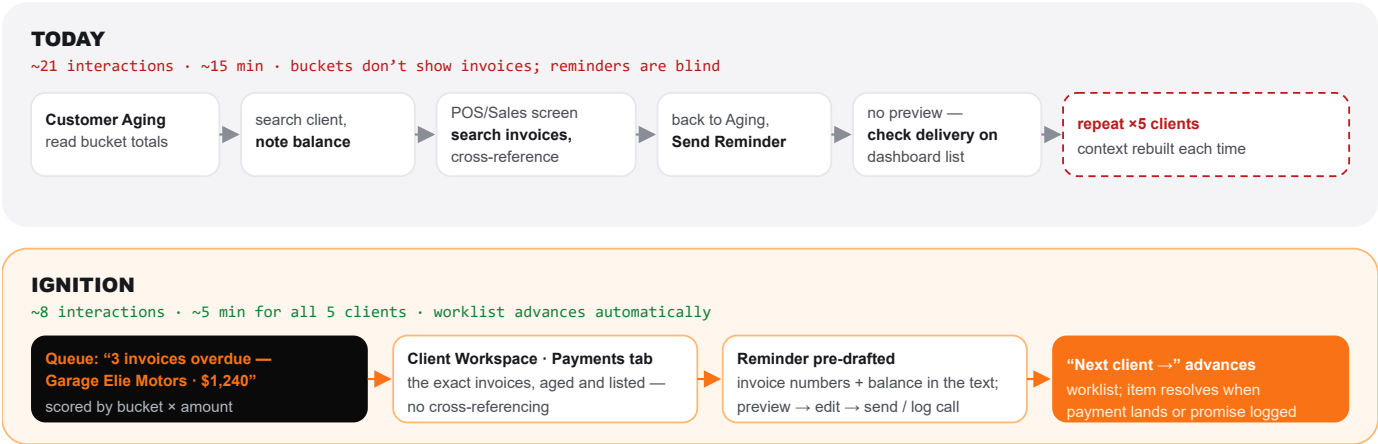


FIG 4 · Swimlane — the daily collections pass. The aging table becomes a self-advancing worklist driven by the Action Queue.

04 The Action Queue as a Workflow Engine

What exists today — the honest baseline

`buildActionQueue()` in `dashboard-view.js` already detects six signals and renders them as cards. That is a genuinely good foundation — and exactly why the redesign keeps the concept and fixes the mechanics:

TODAY'S BEHAVIOR (FROM CODE)	CONSEQUENCE	IGNITION CHANGE
Fixed insertion order; <code>tasks.slice(0, 4)</code> caps the list	A \$12 failed message can display while a \$4,000 receivable is cut off	Score-ranked list, no hard cap; overflow groups by category
Items carry only <code>view: "accounting"</code> etc.	Deep link lands on a whole screen; user re-finds the entity	Deep-link contract carries entity, tab, action, and a pre-drafted payload
Recomputed from scratch on every load; no state	No memory of what was acted on or dismissed; same items nag daily	Persistent item identity: <code>ruleKey + entityId</code> ; lifecycle survives refresh
Aggregate items only ("14 open transactions")	Not actionable without manual triage	Per-entity items ("Garage Elie Motors · 3 invoices · \$1,240"), worklist-able

Prioritization: severity × money at stake

Every item gets a score when signals are evaluated. Score determines order; nothing is silently dropped.

$$\text{score} = S \times (1 + \log_{10}(1 + \text{money_at_stake_usd})) \times A$$

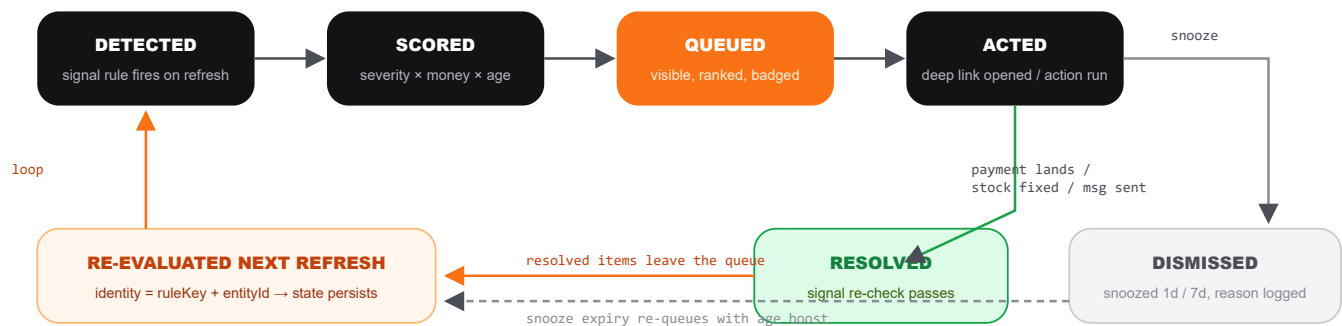
S — severity class: S1 blocking/losing money now = 100 (negative daily P&L, day-close variance) · S2 money waiting = 60 (overdue receivables, quote-ready requests, currency-squeezed margins) · S3 hygiene = 30 (red stock segments, failed messages, expiring quotes) · S4 informational = 10 (no sales yet today).
money_at_stake — the receivable balance, quote value, or margin exposure attached to the entity; items with no monetary dimension use their category's median so they are ordered, not zeroed.
A — age boost: +10% per day unresolved, capped at 2× — items that linger rise, they don't fade.

Worked example: "Garage Elie Motors, 3 invoices 30+ days, \$1,240" → $60 \times (1 + \log_{10}(1241)) \times 1.0 \approx 246$. "1 failed WhatsApp message (\$0 attached, category median \$80)" → $30 \times (1 + \log_{10}(81)) \approx 87$. The receivable outranks the retry, in that order, every time — which matches how the owner actually triages.

Signal catalog and deep-link targets

SIGNAL (RULE KEY)	CLASS	DEEP LINK — WHERE THE CLICK LANDS
Negative daily P&L <code>exists today</code>	S1	Dashboard P&L drill-down: expense breakdown, biggest delta highlighted
Day-close cash variance <code>new</code>	S1	Day-Close card → unmatched receipts list
Overdue receivables per client <code>exists (aggregate)</code>	S2	Client Workspace · payments tab · reminder pre-drafted with the exact invoices
Quote-ready part requests <code>computed, not queued</code>	S2	Request detail with live stock + drafted quote message
Currency-squeezed margins <code>exists today</code>	S2	Repricing sheet filtered to the affected parts, suggested price pre-filled
Red stock segments <code>exists today</code>	S3	Stock view filtered to the red segment, dead-stock action panel open
Failed customer messages <code>exists today</code>	S3	The client's message thread, failed message selected, "Retry" focused
Quotes expiring ≤3 days <code>new</code>	S3	Quote detail with follow-up message drafted
Low stock below min <code>snapshot card today</code>	S3	Reorder Center pre-filtered to the flagged parts
No sales posted yet <code>exists today</code>	S4	POS draft, focused, ready to scan

Item Lifecycle & Deep-Link Contract



contrast with today: buildActionQueue() is stateless – every refresh recomputes, nothing is remembered, acted = never happened

FIG 5 • Action Queue item lifecycle. Persistence is keyed by ruleKey + entityId so acting or snoozing survives refresh and re-login.

The deep-link contract

Every queue item carries a structured target instead of today's bare `view` string:

```
{ view: "client-workspace", entityId: 412, tab: "payments",
  action: "send-reminder", draft: { template: "AgingReminder", invoiceIds: [8811, 8873, 8901] } }
```

- **view + entityId** — lands inside the entity, not on a list screen. “3 invoices overdue for Garage Elie Motors” opens *that client's* workspace, payments tab — never the generic aging table.
- **action + draft** — the target screen opens with the action sheet already populated: reminder text drafted with the exact invoice numbers, payment sheet pre-filled with the balance, reorder list pre-checked.
- **Round trip** — completing the action emits an event (`queue_item_acted`); the next signal evaluation flips the item to RESOLVED if the underlying condition cleared. The queue is therefore a closed loop: it can tell the owner not just what needs doing, but what *got done* today — which becomes the day-close summary.

05 Client Workspace — Operational Flows

The Client Workspace replaces three partial views — Contacts (read-only list, no search, no detail), Customer Aging (buckets without invoices), and per-screen lookups in POS/Sales, Quotes and Part Requests — with one screen per client: identity header (name, phone, WhatsApp status, balance, aging pill), an action bar (**New Invoice · Receive Payment · New Quote · Send Reminder**), and tabs (Overview · Invoices · Payments · Quotes · Requests · Messages · Activity). Four flows carry the daily load:

1 · Receive payment without leaving the screen

Action bar → payment sheet slides over the tab area. Amount defaults to full balance; typing a partial amount shows oldest-first allocation across open invoices, editable per line. Confirm posts the journal entry behind the scenes (same double-entry the manual journal produces today — cash debit, A/R credit), prints a receipt, and the header balance + aging pill update in place. **4 steps, ~30 s**, versus a 12-step manual journal today.

2 · Invoice from client context

“New Invoice” opens the same keyboard-first POS draft as `Ctrl+K inv`, with the client pre-filled — no customer dropdown at all (today’s POS makes the operator scroll a `<select>` capped at 100 rows even when they arrived from a client’s record). On post, the invoice appears at the top of the Invoices tab and the balance updates without a reload.

3 · Aging-driven collection worklist

The Payments tab shows the client’s open invoices *aged inline* (the join the owner does by hand today between customer-aging-view and invoices-view). In worklist mode — entered from the queue item or the aging screen’s “Start collections” — the workspace advances client-to-client sorted by bucket × amount, logging an outcome per client: *reminder sent · promise-to-pay (date) · dispute · unreachable*. Outcomes feed the next day’s queue: a broken promise re-queues at higher score.

4 · Communication follow-ups with templates

The Messages tab is the client’s whole thread: invoices sent, reminders, quote follow-ups — with delivery status per message, replacing today’s blind fire-and-forget (aging’s “Send Reminder” posts `accountBalanceReminder` with no preview; failures surface only on the dashboard). Templates — *AgingReminder, InvoiceCopy, QuoteFollowUp, RequestQuote* — render with live data for preview and edit before send. A failed message shows inline with one-click retry, and raises an S3 queue item pointing back to this exact thread.

What this eliminates, screen by screen

TODAY’S FRAGMENT	GROUNDLED LIMITATION	ABSORBED INTO
<code>contacts-view.js</code>	Two static lists (customers/suppliers), <code>pageSize=100</code> , name + phone + opening balance only; no search, no row click, no create form	Workspace directory with type-ahead; header identity card; inline “new client” (also reachable from the sale flow)
<code>customer-aging-view.js</code>	Buckets and totals, but no invoice drill-down; reminder without preview	Payments tab (aged invoice list) + worklist mode + previewed reminders
<code>invoices-view.js</code> lookups	Finding one client’s invoices means a text search over all invoices; no receive-payment action on rows	Invoices tab, pre-filtered; Receive Payment in the action bar
<code>quotes-view.js</code> lookups	Quotes found by global search; convert and send live on two different screens	Quotes tab with Convert & Send as one previewed action
<code>part-requests-view.js</code> lookups	Requests not linked to the requester’s history or balance	Requests tab beside the client’s invoices and credit position

Design guardrail: the workspace adds no new data-entry concepts. Every action reuses an existing API flow (`/api/sales`, `/api/quotes/{id}/convert`, `/api/communications/send`, journal posting) — it relocates the entry point next to the client, which is where the operator already is mentally.

06 Speed Mechanics

Global **Ctrl** **K** command palette — the grammar

One input, available on every screen, replacing the dashboard’s query-discarding search box. Verbs are 1–4 characters; arguments are fuzzy-matched with type-ahead. Enter always executes the top match; results render inline where the answer is informational (stock/price) instead of navigating.

COMMAND	RESULT
inv eli	New invoice draft, client “Garage Elie Motors” attached, cursor in the scan/code field
inv	Walk-in sale draft (today’s default customer option, minus the dropdown)
pay eli 250	Payment sheet for Elie, \$250 pre-filled, oldest-first allocation shown
part n54 coil / part 11427566327	Inline result card: price, stock by warehouse, margin flag — no navigation (T6 in 2 steps)
q eli	New quote for Elie, same line-builder as invoices
c eli	Open Elie’s Client Workspace (overview tab)
cust + Nabil Fakhry	Inline client create (name + phone), returns to whatever flow invoked it
wa eli	Elie’s message thread with template picker
#8811	Invoice 8811 detail (works for quote and request numbers too)
> reorder	Navigate to any of the registered screens by fuzzy name — the long tail of the 74-screen registry stays reachable without sidebar archaeology

Keyboard-first counter sale

The POS draft is an input loop, not a form: scan field → qty → scan field. Tab/Enter only; the mouse is never required. Price edits are **F4** on the focused line; tender defaults to exact total and cash (today’s free-text “payment method” input becomes a 1-key choice: **Cash** / **Transfer** / on **Account**). Target: 2-line sale in ~30 s (Fig 1).

Barcode flow tie-in

Scanners act as keyboard wedges: a scan anywhere in the palette or the POS scan field resolves by barcode → internal code → OEM code (the same identifiers the dashboard already lists as search tabs). A scan on the dashboard opens the part card; a scan inside a draft adds a line. The existing Barcode Mode screen (`barcode-mode-view.js`) remains for camera-based scanning on mobile.

Recents & favorites

The palette’s empty state shows the last 5 clients, last 5 parts, and pinned actions — because counter traffic is heavily repeat business, most lookups become two keystrokes (**Ctrl+K**, **↓**). Recents are per-user and local-first, so they cost no API round-trip.

Latency budget

Palette matching runs against a cached client + part index (the app already ships the full parts list to the browser via `api.list("/api/parts")` on dashboard and POS load — Ignition reuses that as a versioned cache instead of a per-screen refetch). Budget: <50 ms keystroke-to-result on cached data; stock quantities revalidate async and patch the card.

Why keyboard-first pays for itself: T2 + T6 run ~55 times a day. Every second shaved off their loop saves nearly a minute a day; the palette shaves ~65 s and ~12 s respectively. No other single mechanism in this package moves the daily total more.

07 Cross-Device Workflow

What the owner checks from a phone

The owner’s phone sessions are short, frequent, and read-mostly-plus-approve. The mobile web layout keeps exactly two blocks above the fold:

- **KPI band** — Sales today, Profit today, Cash, Overdue receivables. Four numbers, full-width, tap-to-drill.
- **Action Queue** — the same scored list as desktop. Phone-appropriate actions per item: approve a pre-drafted reminder (one tap — the draft was composed by the deep-link contract, not on the phone), snooze, or “flag for desk” which pins the item to the top of the desktop queue.

Also phone-friendly: receiving a payment (the 4-step sheet works one-handed), checking a part’s stock/price via the palette in search-first mode, and the day-close summary card in the evening.

What stays desktop-only

KEPT ON DESKTOP	WHY
Line-item invoice and quote building	Barcode wedge + numeric keypad + printer live at the counter; multi-line entry on a phone is slower than walking to the desk
Manual journal entries, report builder, schema tools	Dense, error-sensitive, low-frequency — no mobile payoff
Bulk stock operations, reorder confirmation, stock arrival	Physically happens at the shelf/desk with the goods present
Collections worklist (full mode)	Requires the aged-invoice detail view; the phone gets the approve/snooze slice of it

Offline & poor connectivity at the shop counter

Reality: counter connectivity drops in exactly the moments that matter (a queue of customers). The failure mode to design out is “the sale stops because the spinner spins.”

- **Read path is cache-first.** The parts index (already fully client-side today) and the client directory are versioned local caches. Stock lookups and price checks answer instantly from cache with a “as of 10:42” badge; a background revalidation patches quantities when the network returns.
- **Write path is queue-and-sync.** Invoice posts, payment receipts and reminder sends enqueue locally with an optimistic receipt (marked “pending sync”). On reconnect, the queue drains in order; the server remains authoritative — a stock rejection (the same validation `/api/sales` performs today) surfaces as a red queue item on the dashboard rather than a silent loss.
- **Barcode keeps working.** Scans resolve against the local index, so the input loop never blocks on network.
- **Visible state, no mystery.** A connectivity chip in the command bar shows pending-sync count; tapping it lists queued writes. Nothing auto-retries invisibly past 3 attempts without surfacing.
- **Out of scope for offline:** journal entries, conversions, and anything multi-user-sensitive — these fail fast with a clear “needs connection” message instead of queuing.

Cross-device rule of thumb: the phone decides and approves; the desktop composes and executes. Both share one Action Queue state, so an item acted on from the phone is resolved on the desktop and vice versa — one to-do list for the business, not one per device.

08 Adoption & Rollout

Side-by-side period (weeks 1–4)

The new Dashboard and Client Workspace register as additional screens alongside the existing ones — the screen-registry pattern makes this a two-line addition, and legacy screens stay reachable via `Ctrl+K` `> screen-name` throughout. Week 1–2: new screens are opt-in (a “Try the new dashboard” banner). Week 3–4: new screens become the default; legacy screens remain one command away. Removal of the legacy Contacts/Aging screens happens only after the success gate below is met — not on a calendar date.

Telemetry — verifying this report’s projections

Every number in Section 3 becomes a measured distribution. Events are lightweight client-side beacons with `tenantId, userId(role), taskId, ts`:

EVENT	FIRES WHEN	VERIFIES
task_start / task_complete	Task boundary heuristics: e.g. T2 = first palette/POS interaction → invoice posted; T4 = payment sheet open → posted	Time-on-task p50/p90 per task id (T1–T10) — the core before/after claim
screen_view	Every navigation, with source (sidebar / palette / deep-link)	Screens-per-task; navigation share of session time
palette_used	Command executed, with verb + result kind	Adoption of the fast paths; verb frequency tunes the grammar
queue_item_shown / acted / resolved / dismissed	Lifecycle transitions (Fig 5)	Queue precision — are we surfacing things people act on?
reminder_previewed / edited / sent	Messages tab + worklist	Whether previews change content (they should — blind sends are today’s norm)
sync_queue_depth	Offline write queue high-water mark, daily	Whether the offline design carries real counter load

Success metrics — targets to hit before legacy retirement

METRIC	BASELINE (TODAY, EST.)	TARGET (WEEK 6)
Counter sale (T2) time-on-task, p50	~95 s	≤ 35 s
Receive payment (T4) time-on-task, p50	~3 min	≤ 45 s
Stock check (T6) answered without leaving current screen	0%	≥ 80%
Sales initiated via palette or workspace (vs sidebar+dropdown)	0%	≥ 60%
Queue items resolved same-day	n/a (no lifecycle)	≥ 70%
Queue items dismissed as irrelevant	n/a	≤ 15% (precision guard)
Overdue >30d receivables balance	baseline captured wk 0	–20% by day 60
Reminder delivery failures resolved within 24 h	untracked	≥ 90%
Screens visited per session, p50	measured wk 1	–40%

Training & rollback

- **Training is one laminated card:** the palette grammar table from Section 6. The palette also teaches itself — empty state shows the top 6 verbs with examples, and slash-help `( ? )` lists everything.
- **Rollback criteria:** if week-4 p50 for T2 exceeds 60 s, or queue dismissal rate exceeds 30%, the default flips back to legacy while the specific mechanism is reworked. Telemetry keeps flowing either way, so the comparison is honest in both directions.

Figure 6 — One Shop Day, Before vs After

Administrative overhead (navigation, dropdown hunts, cross-referencing, re-keying) across a 08:00–18:00 day for the whole shop, aggregated from the Section 3 table. Selling, stocking and customer conversation are untouched — this is purely the UI tax.

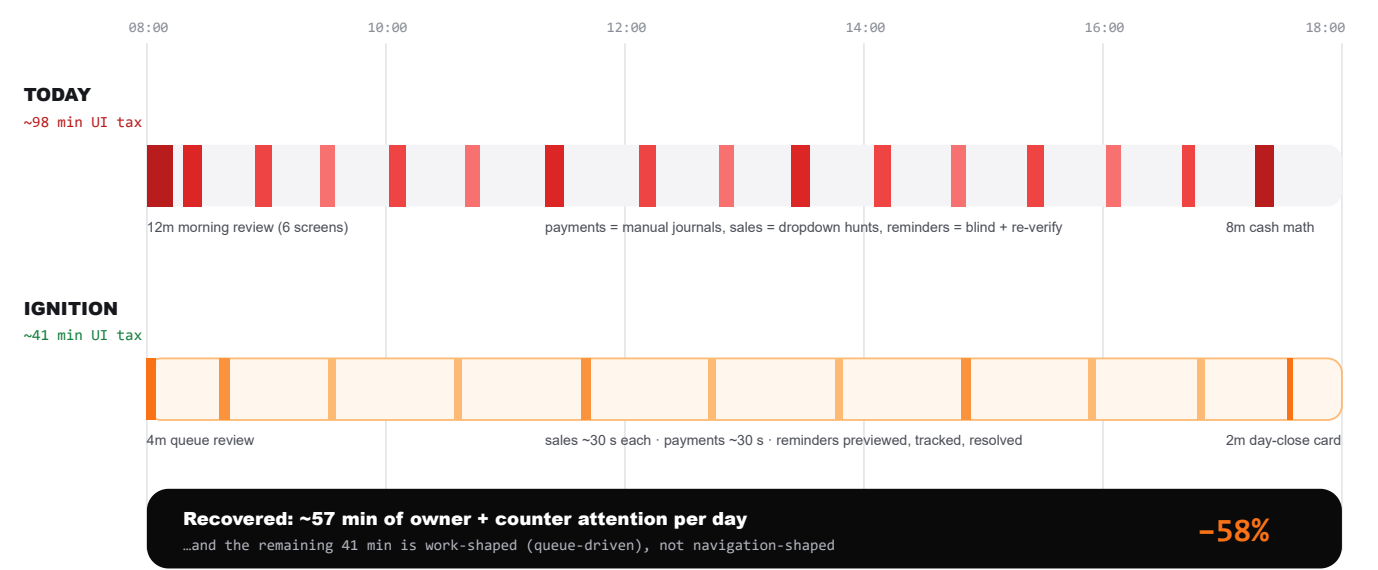


FIG 6 · Administrative overhead across one shop day. Red/orange blocks are minutes spent on UI mechanics rather than customers.

Closing

The redesign wins not by adding capability — nearly every flow in this report re-uses an API the app already has — but by **collapsing distance**: between a signal and its action (Action Queue deep links), between a client and everything about them (Client Workspace), and between an intent and its execution (**Ctrl+K**). The projections total ~98 minutes a day; the telemetry in Section 8 exists so that six weeks after rollout, that number is no longer a projection.

END OF REPORT 05 — NEXT: REPORT 06.